

Cloud-Networking-Assignment-3

Q1. Set up a free-tier AWS account (or use an AWS Cloud sandbox) and launch a basic EC2 instance that serves a static HTML page. Document the steps you took and the public URL of your instance.

Ans:

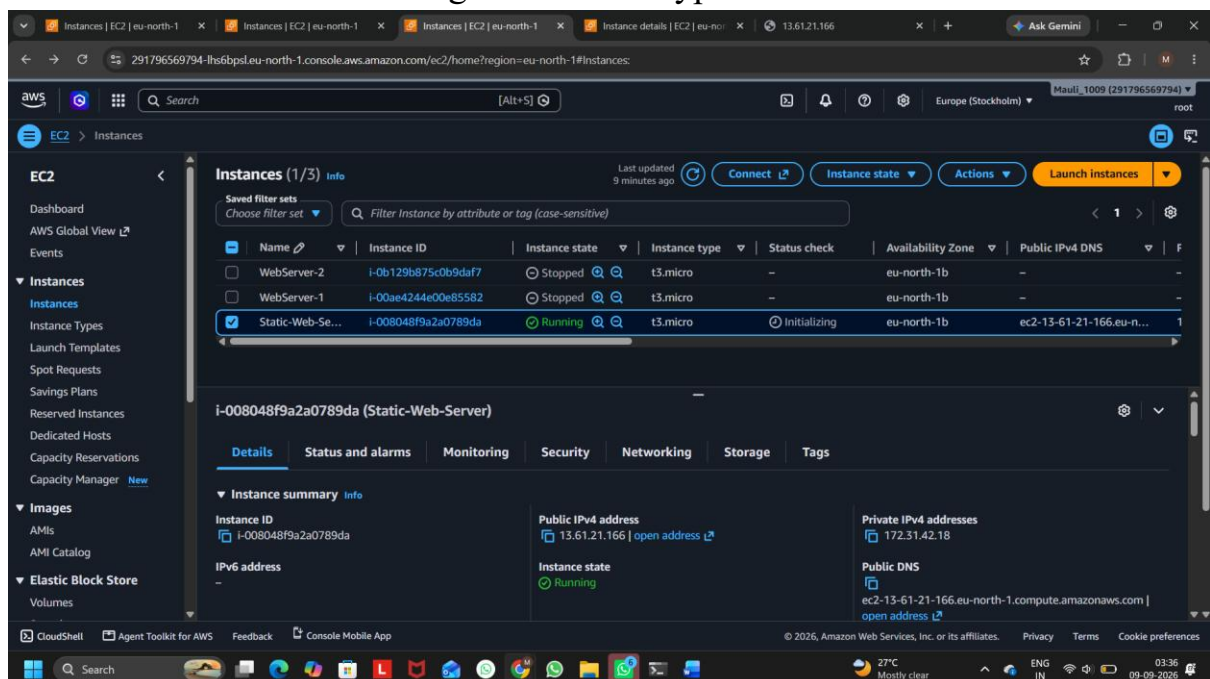
Aim

To launch a basic **EC2 instance** in AWS and configure it to serve a **static HTML page** that can be accessed using its public IP address.

Procedure:

1. Launch an EC2 Instance

The **Amazon EC2** service was opened from the AWS Management Console. A new EC2 instance named **Static-Web-Server** was launched using an **Amazon Linux** AMI and a free-tier eligible instance type.



Screenshot 1: EC2 Instance Running

2. Configure Security Group

The Security Group was configured to allow **HTTP traffic on port 80** from the internet. This allows users to access the web server through a browser.

3. Install and Start Apache Web Server

The EC2 instance was connected using SSH through the Windows CMD. The Apache web server was installed and started using:

```
sudo dnf install httpd -y
```

```
sudo systemctl start httpd
sudo systemctl enable httpd
```

4. Create the Static HTML Page

A simple static HTML page was created in the Apache web server's document root:

```
echo "<h1> Hello from Mauliiiiiii</h1>" | sudo tee /var/www/html/index.html
```

The page was stored in:

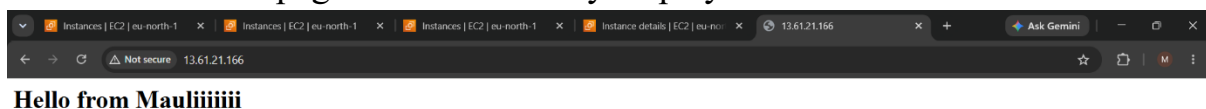
/var/www/html/index.html

5. Access the Web Page

The **Public IPv4 address** of the EC2 instance was used in a web browser with HTTP:

http:// 13.61.21.166

The static HTML page was successfully displayed in the browser.



Screenshot 2: Static HTML Page Served by EC2

Public URL: http:// 13.61.21.166

Q2.Create an Auto Scaling Group in AWS that maintains a minimum of 1 and a maximum of 3 EC2 instances for your web server. Set the scaling policy to add a new instance if CPU utilization goes above 60% for 5 minutes. Hint: Use the AWS Management Console's Auto Scaling Groups section and attach your EC2 launch template or configuration.

Ans:

Aim

To create an **Auto Scaling Group (ASG)** in AWS that maintains a minimum of **1 EC2 instance** and a maximum of **3 EC2 instances**, and automatically adds a new instance when **CPU utilization goes above 60% for 5 minutes**.

Procedure:

Step 1: Create a Security Group

First, I created a security group named **WebServer-SG** for the web server.

The following inbound rules were configured:

- **SSH – Port 22** → My IP
- **HTTP – Port 80** → Anywhere IPv4

The default outbound rule was kept unchanged.

Step 2: Create a Launch Template

I created a launch template named **WebServer-Launch-Template**.

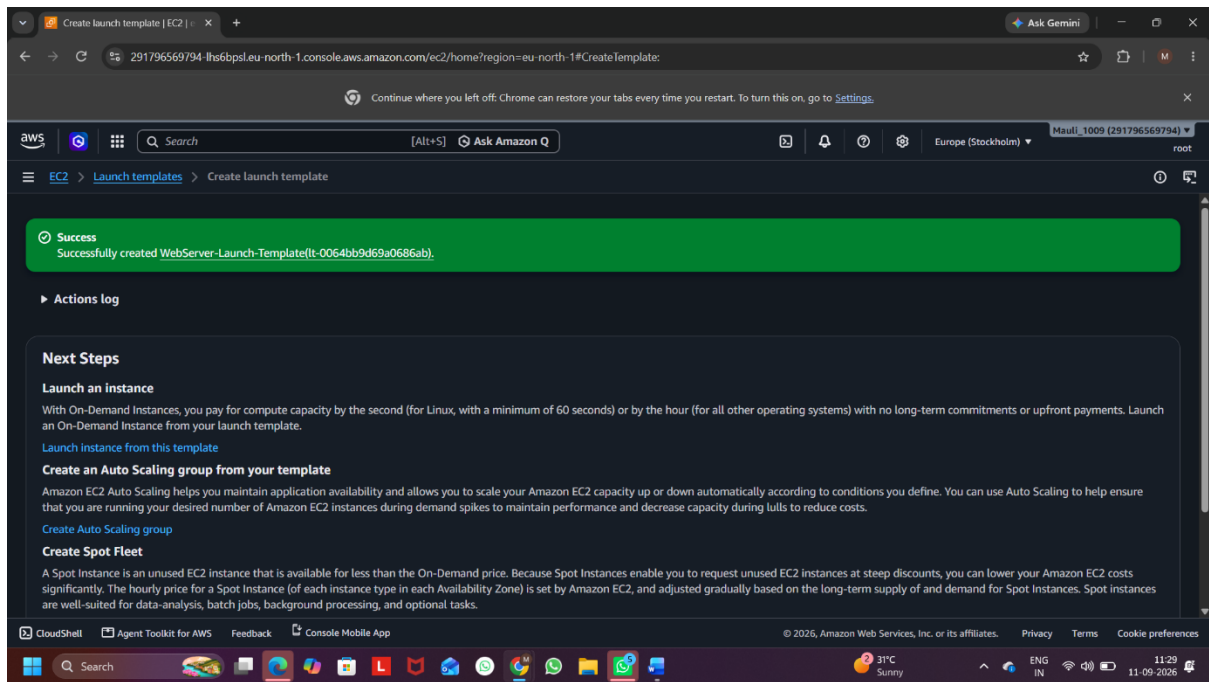
The following configuration was used:

- **AMI:** Amazon Linux 2023
- **Instance Type:** t3.micro
- **Key Pair:** My-web-server-key
- **Security Group:** WebServer-SG
- **Storage:** Default 8 GiB gp3
- **Subnet:** Not included in the launch template

For the web server, the following User Data script was added:

```
#!/bin/bash
dnf update -y
dnf install -y httpd
systemctl enable httpd
systemctl start httpd
echo "Hello from Auto Scaling Web Server" > /var/www/html/index.html
```

This script automatically installs and starts the Apache web server when a new EC2 instance is launched.



Screenshot 1: successfully created and configured template

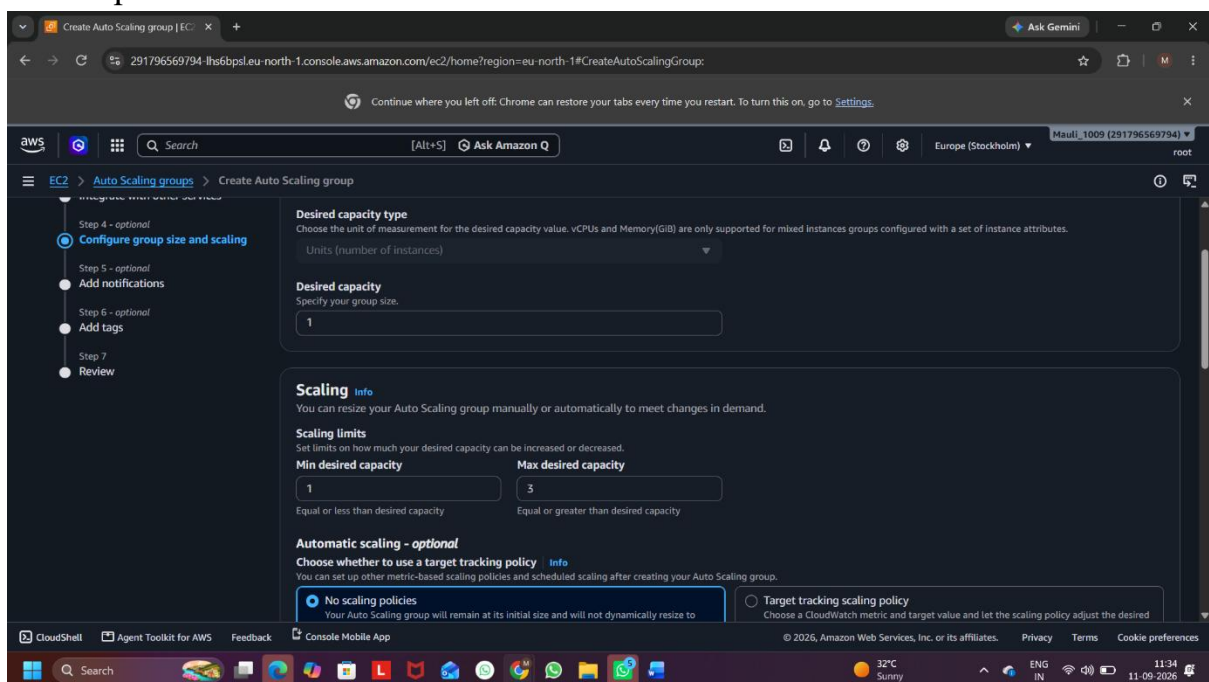
Step 3: Create the Auto Scaling Group

I created an Auto Scaling Group named **WebServer-ASG** and attached the **WebServer-Launch-Template** to it.

The Auto Scaling Group was configured with:

- **Desired capacity: 1**
- **Minimum capacity: 1**
- **Maximum capacity: 3**

The required VPC and subnet were selected.



Screenshot 2: Auto Scaling Group Capacity

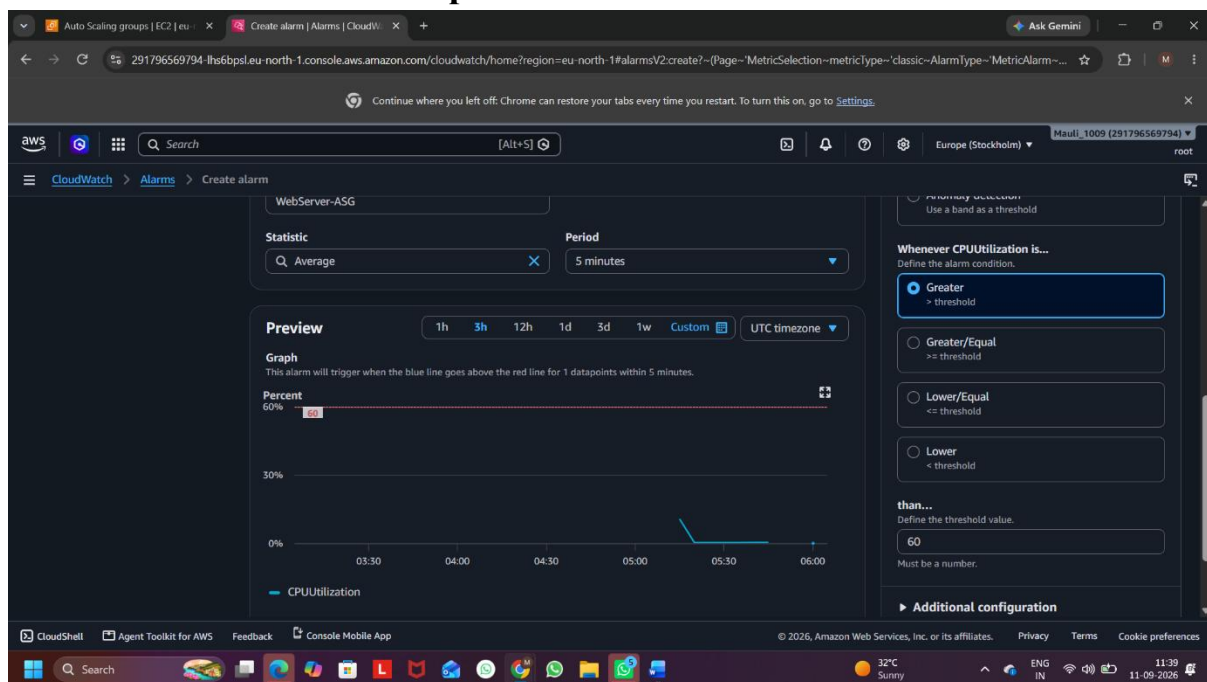
Step 4: Create a CloudWatch Alarm

I created a CloudWatch alarm named **CPU-High-60-5min**.

The alarm was configured as:

- **Metric:** CPUUtilization
- **Auto Scaling Group:** WebServer-ASG
- **Statistic:** Average
- **Period:** 5 minutes
- **Condition:** Greater than 60%
- **Evaluation:** 1 out of 1 datapoint

Therefore, the alarm is triggered when the average CPU utilization is **greater than 60% for one 5-minute period**.



Screenshot 3: CloudWatch Alarm – CPU Above 60% for 5 Minutes

Step 5: Create a Dynamic Scaling Policy

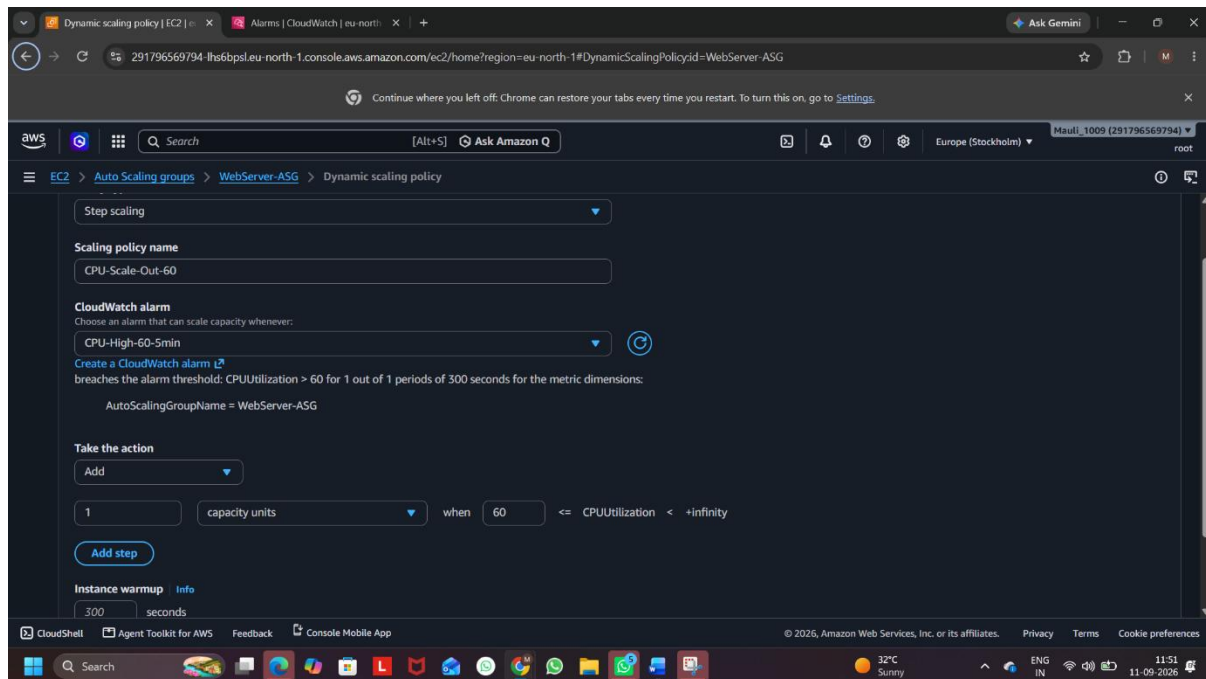
A **Step Scaling** policy named **CPU-Scale-Out-60** was created for the Auto Scaling Group.

The CloudWatch alarm **CPU-High-60-5min** was selected.

The scaling action was configured as:

- **Action:** Add
- **Capacity:** 1 capacity unit
- **Instance warmup:** 300 seconds

Therefore, when the CloudWatch alarm is triggered, the Auto Scaling Group adds **1 new EC2 instance**.



Screenshot 4: Step Scaling Policy – Add 1 Capacity Unit

Result:

The **Auto Scaling Group** was **successfully created** with a minimum capacity of **1** and a maximum capacity of **3** EC2 instances. A **CloudWatch alarm** was configured to monitor CPU utilization. When CPU utilization goes **above 60% for 5 minutes**, the **Step Scaling policy** adds **1 new EC2 instance** automatically.

Q3.Simulate increased traffic to your web server by using an online load testing tool (such as Loader.io or k6) and observe if the Auto Scaling Group launches additional instances when CPU usage spikes.

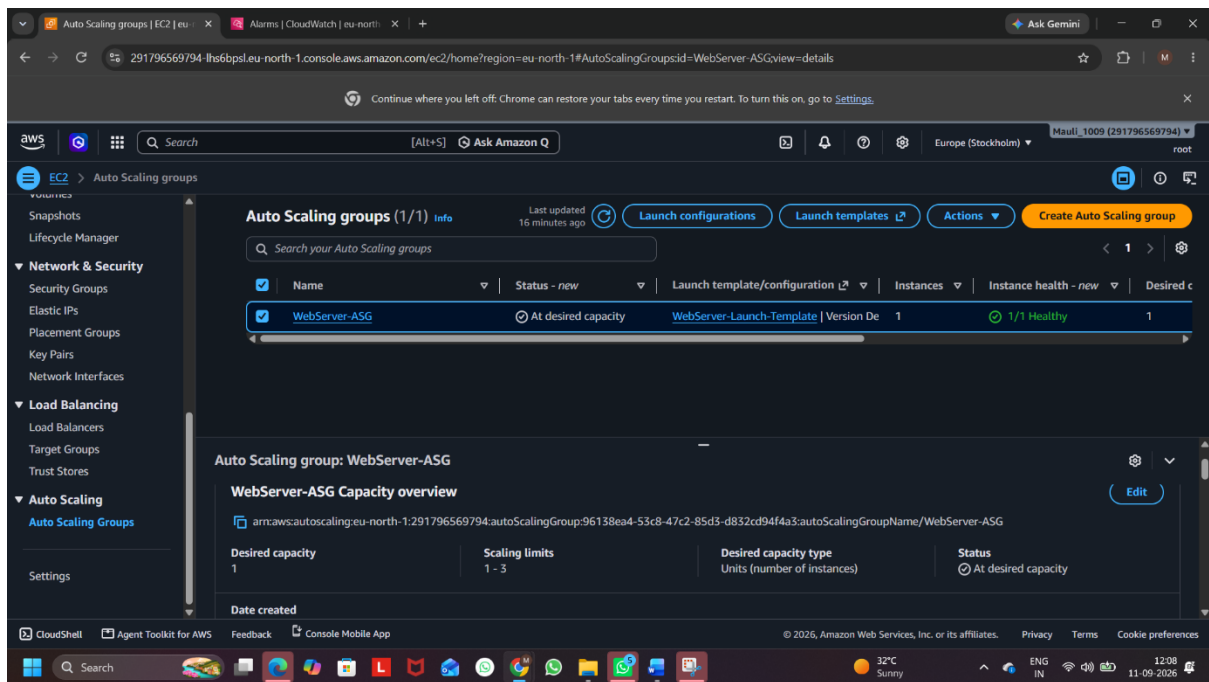
Ans:

Aim:

To simulate increased traffic on the web server using Loader.io and observe the behavior of the Auto Scaling Group.

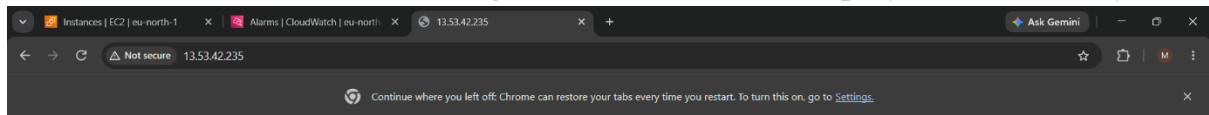
Procedure:

1. The Auto Scaling Group WebServer-ASG was checked before the test. It had **1 desired instance**, with a minimum of **1** and maximum of **3** instances.



Screenshot 1: Auto Scaling Group Before Load Test

2. The web server was opened using its public IP address, and the message **“Hello from Auto Scaling Web Server”** was displayed successfully.

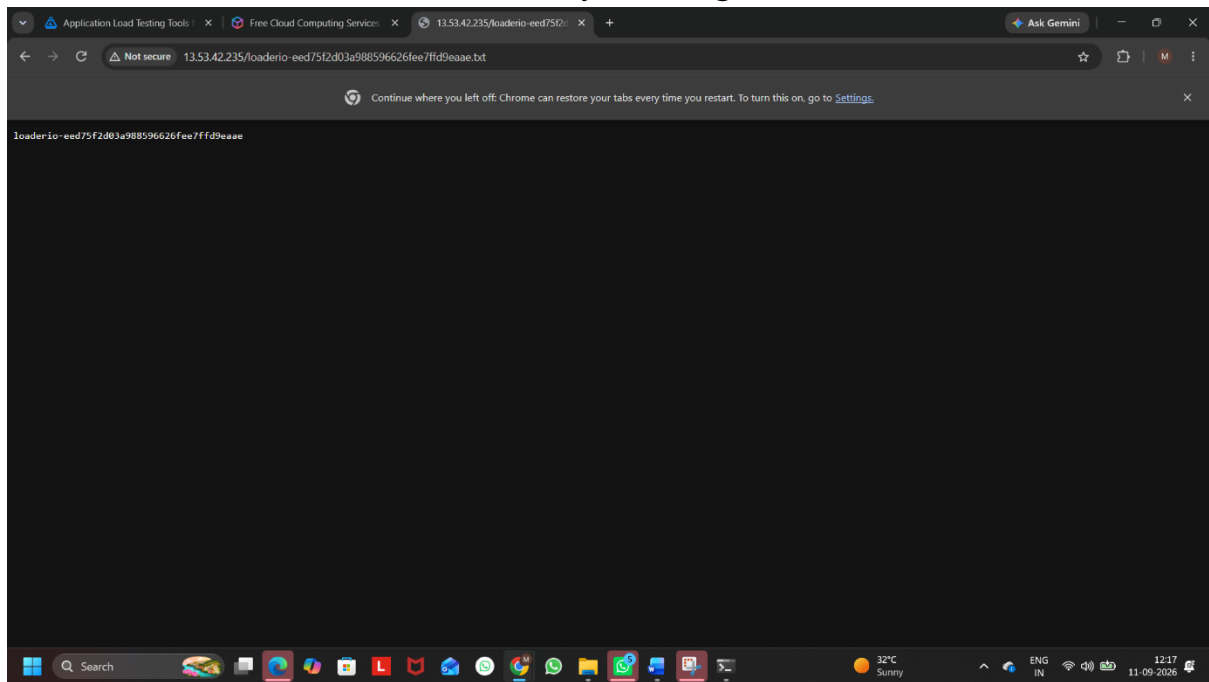


Hello from Auto Scaling Web Server



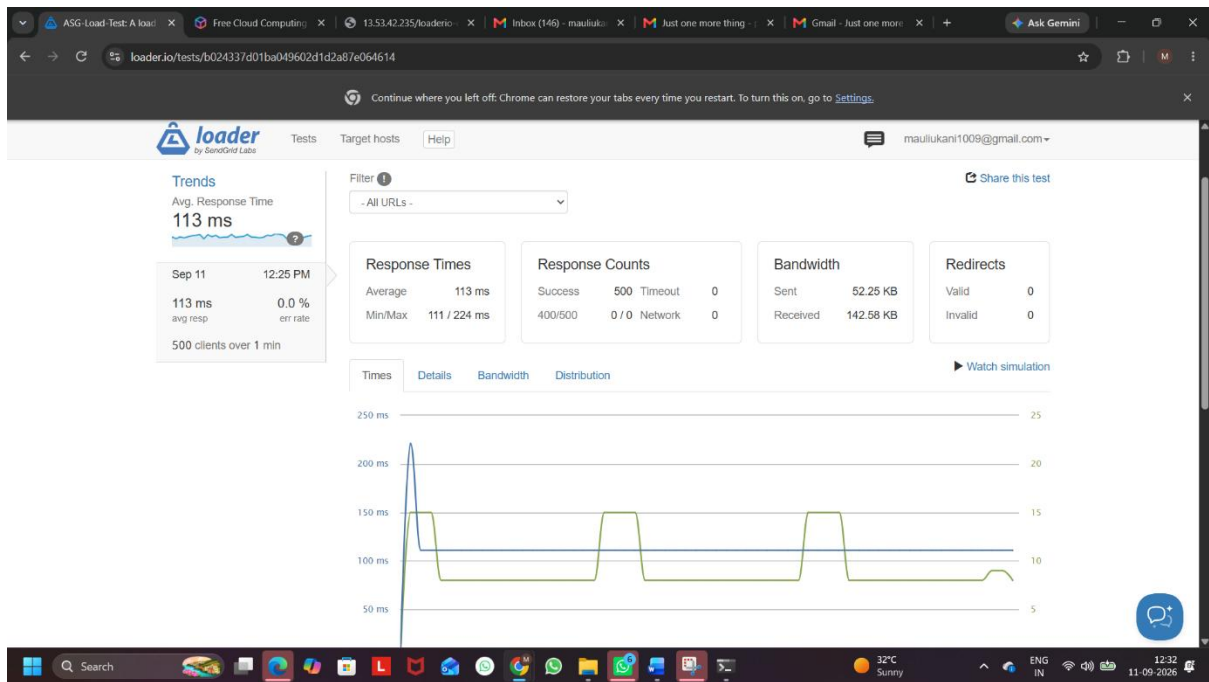
Screenshot 2: Web Server Running

3. Loader.io was used as the online load testing tool. The EC2 public IP address was verified successfully as a target host.



Screenshot 3: Loader.io Host Verification

4. A load test named **ASG-Load-Test** was created with:
- Test type: Clients per test
 - Clients: **500**
 - Duration: **1 minute**
 - Method: **GET**
 - Protocol: **HTTP**
 - Target: 13.53.42.235
5. The load test was started to simulate increased traffic.
6. The test completed successfully with **500/500 successful requests, 0% error rate**, and an average response time of **113 ms**.



Screenshot 4: Loader.io Load Test Results

Result:

The load test successfully generated **500 client requests**, but the Auto Scaling Group **did not launch an additional EC2 instance** during this test.

This is because the configured scaling policy requires **CPU utilization to remain above 60% for 5 minutes**. The Loader.io test was performed for only **1 minute**, so the 5-minute scaling condition was not met.

Q4. After scaling up, reduce the simulated load and verify that the Auto Scaling Group scales the number of instances back down to the minimum. Take screenshots of the scaling activity history as evidence.

Ans:

Procedure:

Step 1: Create Launch Template

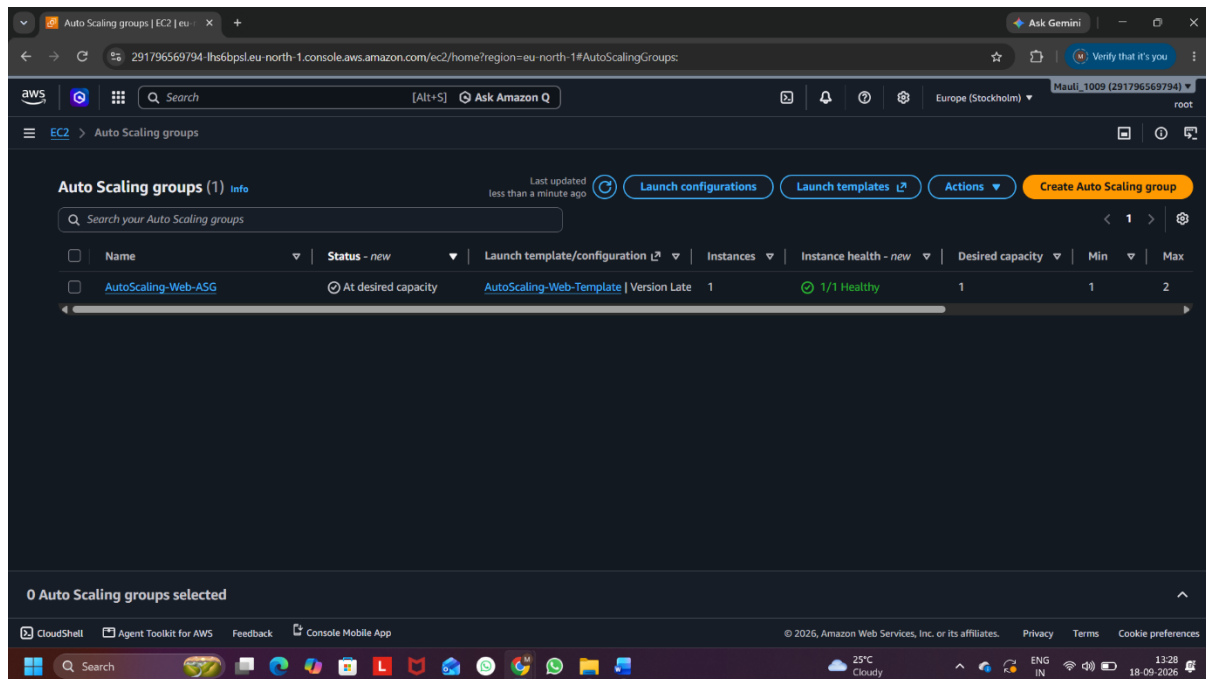
First, a Launch Template named **AutoScaling-Web-Template** was created using **Amazon Linux 2023**. The required key pair and security group were selected, and user data was added to install and start the Apache web server.

Step 2: Create Auto Scaling Group

An Auto Scaling Group named **AutoScaling-Web-ASG** was created using the **AutoScaling-Web-Template**. The capacity was configured as:

- **Minimum capacity:** 1
- **Desired capacity:** 1
- **Maximum capacity:** 2

A **Target Tracking Scaling Policy** was also configured with **Average CPU Utilization = 50%**.



Screenshot 1: Auto Scaling Group Created with Initial Capacity of One Instance

Step 3: Generate High CPU Load

The EC2 instance was connected through SSH using Windows Command Prompt. The following command was executed to generate high CPU utilization:

```
yes > /dev/null &
```



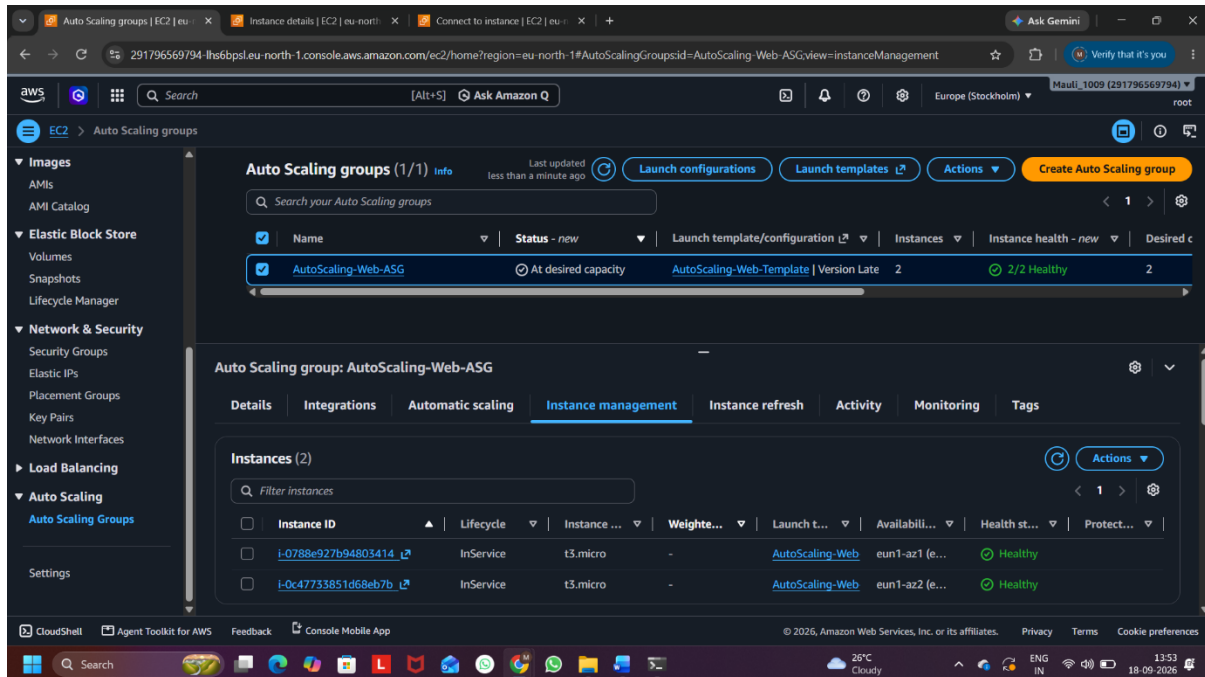
Due to the increased CPU utilization, the Auto Scaling Group automatically launched another EC2 instance and increased the capacity from **1 to 2 instances**.

The screenshot shows the AWS Management Console interface for the 'Auto Scaling groups' page. The top navigation bar includes the AWS logo, a search bar, and various utility icons. The left sidebar contains a navigation menu with categories like 'Images', 'Elastic Block Store', 'Network & Security', 'Load Balancing', and 'Auto Scaling'. The main content area displays the 'Auto Scaling groups (1/1)' page, showing a table with columns for Name, Status, Launch template/configuration, Instances, Instance health, and Desired capacity. The 'AutoScaling-Web-ASG' group is listed with a status of 'Healthy' and 2 instances. Below the table, the 'Auto Scaling group: AutoScaling-Web-ASG' details are shown, including a timeline of events such as 'Launching a new EC2 instance' and 'Target Tracking policy Target Tracking Policy changing the desired capacity from 1 to 2'.

Screenshot 4: Activity History Showing Scale-Out from 1 to 2 Instances

Step 5: Verify Two Instances

The **Instance Management** section was checked and both EC2 instances were shown as **InService** and **Healthy**.

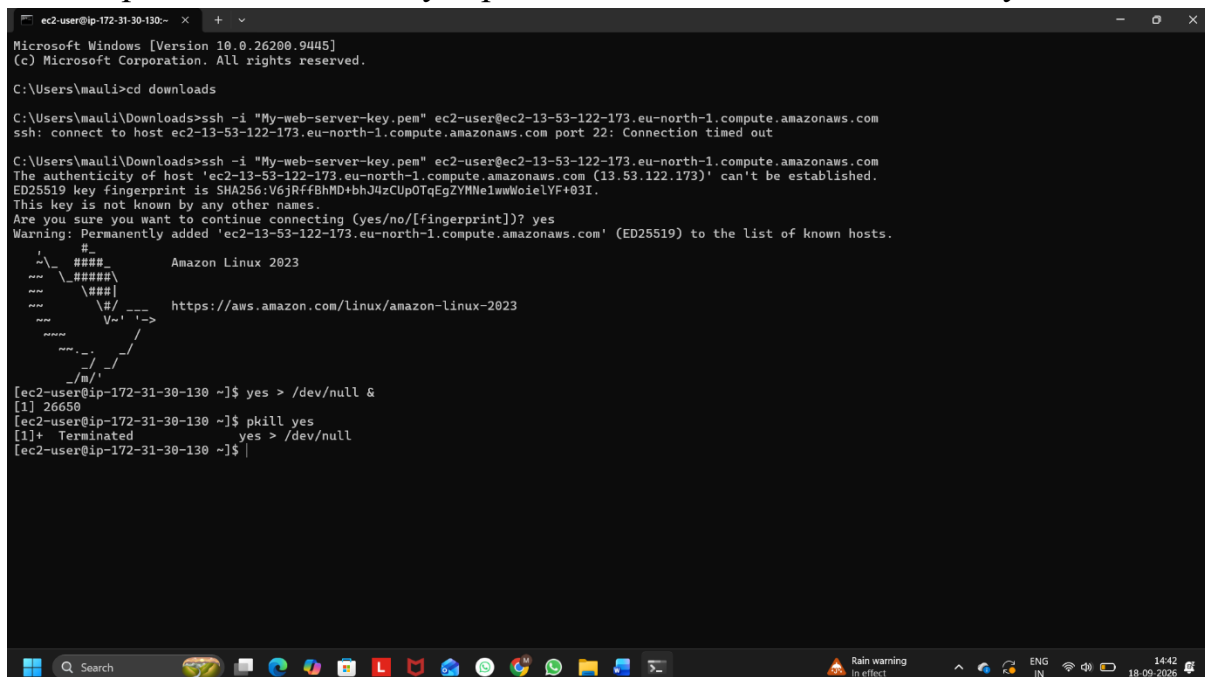


Screenshot 5: Auto Scaling Group with Two Healthy Instances

Step 6: Reduce the CPU Load

After verifying the scale-out, the simulated CPU load was stopped using the command: **pkill yes**

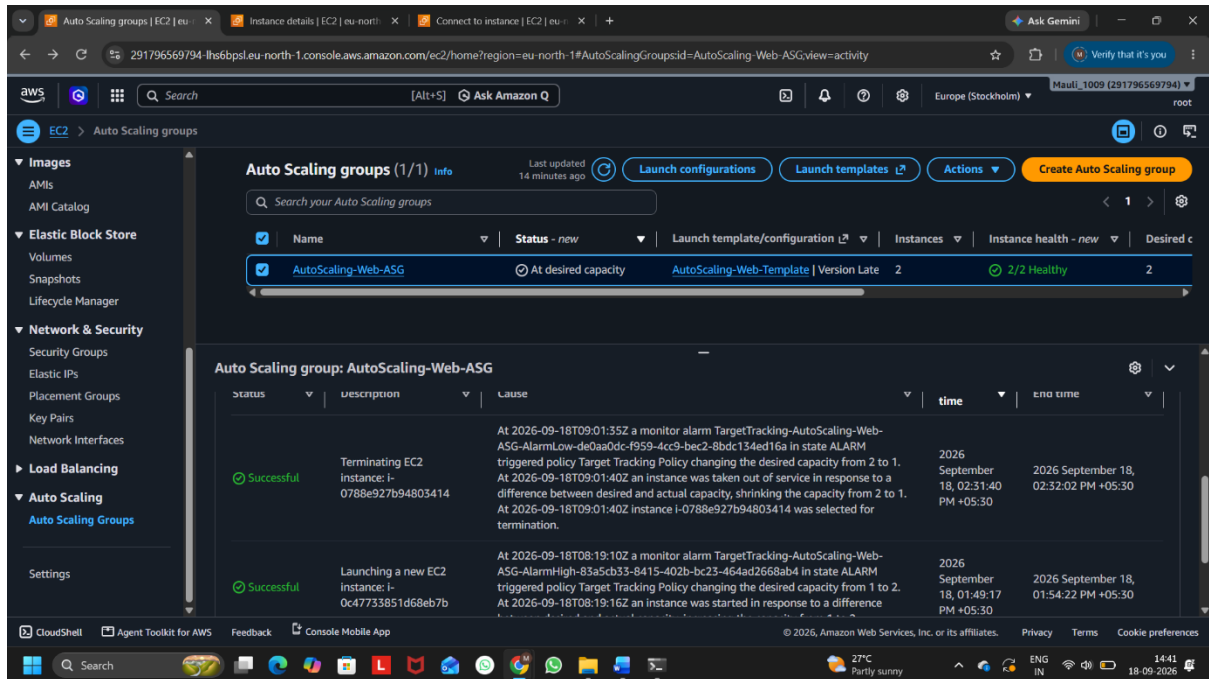
The output showed that the yes process was terminated successfully.



Screenshot 6: Stopping the Simulated CPU Load

Step 7: Verify Scale-In

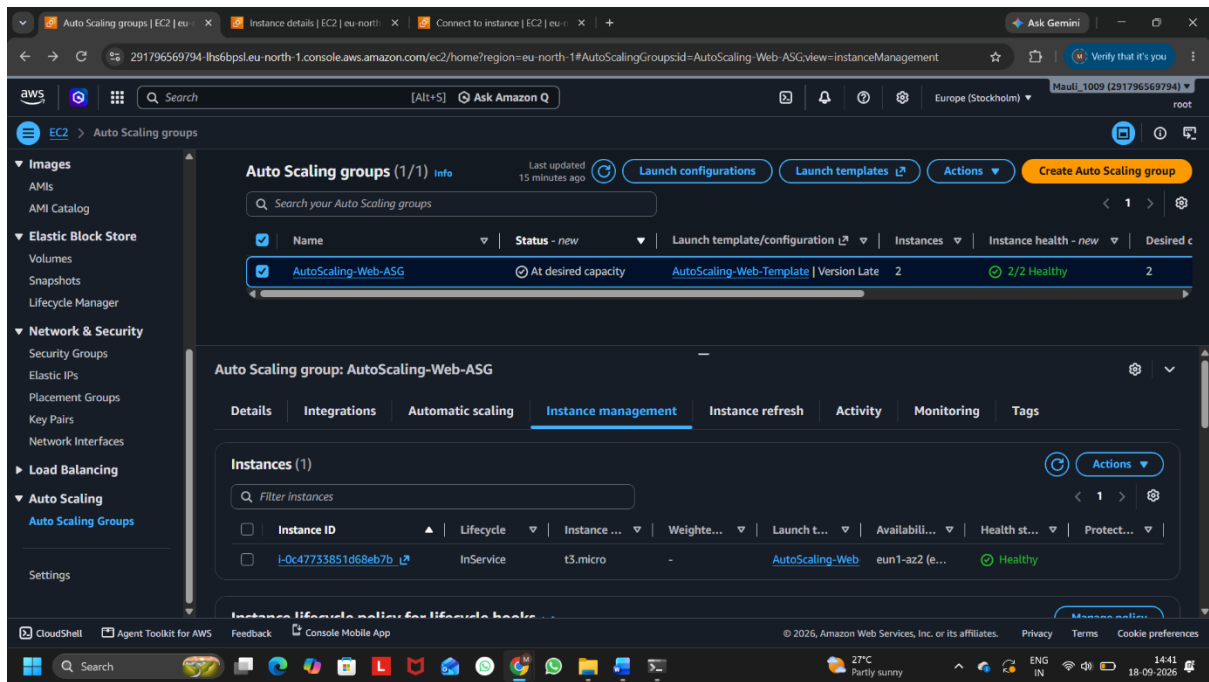
After the CPU utilization decreased, the Auto Scaling Group automatically terminated one EC2 instance and reduced the capacity from **2 to 1 instance**. The successful scale-in operation was verified from the **Activity History**.



Screenshot 7: Activity History Showing Scale-In from 2 to 1 Instance

Step 8: Verify Final Capacity

Finally, the **Instance Management** section was checked. Only **1 healthy EC2 instance** remained, which matched the minimum capacity of the Auto Scaling Group.



Screenshot 8: Auto Scaling Group Scaled Down to Minimum Capacity

Result:

The Auto Scaling Group successfully scaled out from 1 to 2 instances when CPU utilization increased and scaled back in from 2 to 1 instance after the simulated load was reduced. Thus, the Auto Scaling Group returned to its configured minimum capacity.

Q5. Use ChatGPT or another AI tool to generate a list of 3 real-world scenarios (from apps like Zomato, Flipkart, or BookMyShow) where auto-scaling is essential, and explain in your own words why auto-scaling helps in each scenario.

Ans:

3 Real-World Scenarios Where Auto-Scaling Is Essential:

1. Zomato – Lunch/Dinner Time

During lunch and dinner hours, many people order food at the same time.

Zomato also experiences large traffic spikes during special occasions like New Year and Diwali.

Why auto-scaling helps:

Auto-scaling can automatically add more servers when traffic increases, so the app remains fast and does not crash. When traffic becomes normal, extra servers can be removed to reduce cost.

2. Flipkart – Big Billion Days Sale

During the Big Billion Days sale, Flipkart gets a huge increase in users and orders. Flipkart has reported very large traffic spikes during these sales.

Why auto-scaling helps:

Auto-scaling provides more computing resources during the sale to handle searches, product views, carts, and orders. After the sale, the extra resources can be reduced, saving money.

3. BookMyShow – Popular Movie Release

When tickets for a highly popular movie or event go on sale, thousands of users may try to search for shows and book tickets at nearly the same time.

Why auto-scaling helps:

Auto-scaling can increase server capacity during the sudden booking rush. This helps users search shows and complete payments without the website becoming slow or unavailable.